DIGITAL EGYPT PIONEERS INITIATIVE (DEPI)

GRADUATION PROJECT DOCUMENTATION

Track: Data Engineering
Project Title: E-Commerce Data Integration & Analytics Pipeline
GitHub Repository: https://github.com/mth1346686/E-commerce

Team Members & Assigned Roles:

1. **Mahmoud Tarek Mahmoud Hafez (Team Leader, Data Architect & Pipeline Lead):** Overall project management and timeline supervision. High-level system architecture and component design. Designing and implementing the relational dimensional model (Star Schema: Fact and Dimension tables). Developing the core Apache Airflow DAG orchestration, retry logic, and pipeline scheduling.
2. **Mohamed Ahmed Fathy El-Deeb (Senior Data Ingestion & Extraction Engineer):** Designing and building multi-source automated extraction pipelines. Developing REST API consumption modules, dynamic web scraping scripts, and large-scale CSV/JSON file ingestion. Implementing source schema verification and initial payload validation checks.
3. **Mahmoud Abdelnaby Refaat Ibrahim (Senior Data Transformation & Warehouse Engineer):** Designing and developing comprehensive ETL data cleaning and transformation modules using Python and Pandas. Handling missing data imputation, deduplication algorithms, and format standardizations. Implementing relational joins, incremental loading logic, and building analytical SQL Views and database performance indexes.
4. **Mohamed Rezk (Associate Quality Assurance):** Assisting in running predefined automated data-quality assertions and integrity checks. Maintaining code documentation, compiling execution logs, and documenting schema dictionaries. Assisting the engineering team in testing runs, preparing report assets, and presentation slides.

1. Project Planning & Management

1.1 Project Proposal (Overview, Objectives & Scope)

- **Problem Statement:** Modern e-commerce organizations receive operational data (products, customers, transactions, orders) across multiple disjointed sources: external REST APIs, dynamic product web pages, and raw CSV/JSON files. This incoming data frequently suffers from duplicate rows, inconsistent schemas, missing values, and evolving record structures, which prevents business analysts from having a centralized, single source of truth for reporting and strategic decisions.
- **Project Objective:** To design, build, and deploy an automated, end-to-end data pipeline that extracts raw data across diverse formats, validates and cleanses incoming feeds, transforms and loads the structured data into a relational data warehouse, and establishes a robust star schema optimized for analytical reporting.
- **Project Scope:**

- **In-Scope:**
  - Ingesting product catalogs via REST APIs and compliant web scraping.
  - Ingesting customer and transaction records via static/streaming CSV and JSON files.
  - Pre-ingestion validation, deduplication, schema alignment, and imputation using Pandas.
  - Star schema data modeling comprising Fact_Sales, Dim_Customer, Dim_Product, and Dim_Date.
  - Implementing incremental and append-based data loading mechanisms.
  - Creating performance-optimized database indexes and business-focused SQL Views.
  - End-to-end orchestration using Apache Airflow DAGs (Extract -> Validate -> Transform -> Load -> Quality Check) with automated retries and alerting.
  - Packaging the pipeline via Docker containerization and establishing GitHub Actions CI/CD workflows.
- **Out-of-Scope:**
  - Frontend customer checkout applications and real-time payment gateway processing.

1.2 Project Plan (Timeline & Milestones)

Phase	Duration	Milestones & Key Deliverables
Phase 1: Planning & Requirements	Oct 2 – Oct 16, 2026	Finalized project charter, team roles, requirements document, and GitHub repository setup.
Phase 2: System Analysis & Modeling	Oct 16 – Nov 6, 2026	Conceptual architecture diagram, ERD design, DFD mapping, and star schema modeling.
Phase 3: ETL & Pipeline Development	Nov 6 – Nov 30, 2026	Python ingestion modules, transformation scripts, SQL DDL/DML, and functional Airflow DAGs.
Phase 4: Testing, QA & Final Delivery	Nov 30 – Dec 4, 2026	Data quality assertion tests, Docker containerization, final presentation, infographic, and project reports.

1.3 Task Assignment & Responsibilities

- **Mahmoud Tarek Mahmoud Hafez (Core Leadership & Architecture):** Leads system architecture roadmap, Kimball dimensional modeling, DDL schema generation, Airflow DAG workflow implementation, and GitHub governance.
- **Mohamed Ahmed Fathy El-Deeb (Core Extraction & Ingestion):** Develops and maintains all data ingestion scripts, external API connectors, parsing routines, schema validation logic, and raw landing stage operations.
- **Mahmoud Abdelnaby Refaat Ibrahim (Core Transformation & Data Warehouse):** Develops end-to-end data cleansing scripts, complex Pandas business transformations, SQL aggregation views, indexing strategies, and incremental staging loads.
- **Associate Team Member 4 (Support & QA Documentation):** Executes predefined

data-quality check scripts, documents technical summaries and data dictionaries, monitors pipeline run logs, and assists with presentation deliverables.

1.4 Risk Assessment & Mitigation Plan

- **Risk 1: External API rate limiting or temporary server outages.**
  - *Mitigation:* Implement exponential backoff, circuit-breaker logic, and Airflow automated retries paired with cached staging storage.
- **Risk 2: Sudden schema drift or corrupt records in source CSV/JSON feeds.**
  - *Mitigation:* Implement schema validation checks at ingestion; route malformed records to a quarantined dead-letter table without failing the main pipeline.
- **Risk 3: Performance bottlenecks during heavy analytical reporting queries.**
  - *Mitigation:* Apply composite indexing on foreign keys and date dimensions, and materialize high-frequency aggregation queries via database views.

1.5 Key Performance Indicators (KPIs)

- **Pipeline Execution SLA:** Scheduled daily DAG run completion time under 15 minutes.
- **Data Quality & Integrity Rate:** Above 99.5% validity across primary keys, foreign keys, and non-nullable transaction values.
- **Query Latency:** Sub-second execution time (< 1.0s) for standard analytical reports executing on the relational star schema.
- **System Resilience:** Automated recovery and idempotency ensuring zero duplicate transactions upon job retries.

2. Literature Review

2.1 Comparative Analysis of Existing Solutions

Existing industry approaches to managing multi-source sales analytics typically follow two paradigms:

1. **Ad-hoc Cron Scheduling with Standalone Scripts:**
   - *Strengths:* Simple to build initially; minimal infrastructure requirements.
   - *Limitations:* Lacks visual DAG dependency management, offers poor failure alerts, risks data duplication on retry, and lacks centralized logging.
2. **Fully-Managed Proprietary SaaS (e.g., Fivetran + Snowflake):**
   - *Strengths:* Rapid out-of-the-box configuration with zero infrastructure overhead.
   - *Limitations:* High recurring licensing costs, potential vendor lock-in, and limited customization for custom business transformation logic.

2.2 Proposed Solution & Enhancements

- **Enterprise Orchestration via Apache Airflow:** Introduces modular Directed Acyclic Graphs (DAGs) providing visual pipeline state monitoring, programmatic dependency execution, and automated failure handling.
- **Custom Extensibility:** Employs modular Python and Pandas scripts for customized cleansing workflows before committing data to the warehouse.
- **Incremental Processing:** Utilizes watermark-based incremental loading to minimize compute overhead and avoid unnecessary full-table reloads.
- **References:**

- Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*.
- Apache Airflow Documentation: *Architecture, Scheduling & Orchestration Best Practices*.
- McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*.

3. Requirements Gathering

3.1 Stakeholder Analysis

- **Commercial & Sales Analysts:** Require clean, integrated, and reliable transaction records to evaluate KPIs, product performance, and sales trends without manual cleanup.
- **Inventory & Operations Managers:** Rely on accurate stock and order volume metrics to streamline replenishment cycles.
- **Data Engineering & Operations Team:** Require an observable, automated pipeline with logging, clear schemas, and easy disaster recovery.

3.2 User Stories & Use Cases

- **User Story 1 (Business Analyst):** "As a sales analyst, I want to query a pre-aggregated SQL view of daily revenue by product category, so that I can generate executive performance decks without writing complex joins each time."
- **User Story 2 (Operations Lead):** "As an operations manager, I want the system to automatically load the previous day's orders into the warehouse by 6:00 AM, so that operational reporting is ready before business hours begin."
- **User Story 3 (Data Engineer):** "As an engineer, I want the ingestion job to catch malformed JSON payloads and alert the team via Airflow logs while continuing valid transformations, so that processing isn't stalled by bad source records."

3.3 Functional Requirements

1. **Multi-Source Data Ingestion:**
   - Ingest structured customer and order records from CSV and JSON formats.
   - Ingest product catalog attributes via RESTful endpoints or web scrapers.
2. **Data Cleansing & Validation:**
   - Detect and eliminate duplicate transactions based on business keys.
   - Handle missing numerical/text attributes and standardize datetime fields into uniform UTC timestamps.
3. **Data Transformation & Modeling:**
   - Merge external customer data with transactional streams using Pandas.
   - Populate a Star Schema comprised of Fact_Sales, Dim_Customer, Dim_Product, and Dim_Date.
4. **Loading & Optimization:**
   - Support append-only and incremental data loading to prevent duplicates.
   - Create optimized SQL Views for frequent business intelligence queries.
   - Create indexes on foreign keys and dimension filter columns.
5. **Orchestration & Workflow Management:**
   - Execute tasks sequentially in Airflow: Extract -> Validate -> Transform -> Load -> Quality Check.

- Configure retry parameters and error logging mechanisms.